Implement A* Search Algorithm:

i) Using missplaced tiles as Heuristic function:

```
import heapq

def print_grid(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()

def misplaced_tiles(state, goal):
    count = 0
    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            count += 1
    return count

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            new_state = list(state)
            new_state[zero], new_state[new_pos] = (
                new_state[new_pos], new_state[zero]
            )
            neighbors.append(tuple(new_state))

    return neighbors

def astar(initial, goal):
    pq = []
    h = misplaced_tiles(initial, goal)
    heapq.heappush(pq, (h, 0, initial))
```

```
    parent = {initial: None}
    cost = {initial: 0}

    while pq:
        f, g, current = heapq.heappop(pq)

        if g != cost[current]:
            continue

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()

            print("\nGoal reached!")
            print("Total moves:", len(path) - 1)
            print("Total steps:", len(path))

            for step, state in enumerate(path):
                g_value = step
                h_value = misplaced_tiles(state, goal)
                f_value = g_value + h_value

                print("Step:", step)
                print("g(n) =", g_value)
                print("h(n) =", h_value)
                print("f(n) =", f_value)
                print_grid(state)

            return

        for neighbor in get_neighbors(current):
            new_g = g + 1

            if neighbor not in cost or new_g < cost[neighbor]:
                cost[neighbor] = new_g
                parent[neighbor] = current

                h = misplaced_tiles(neighbor, goal)
                new_f = new_g + h

                heapq.heappush(
```

```
                    pq, (new_f, new_g, neighbor)
                )

    print("No solution found.")

initial = tuple(map(int, input(
    "Enter initial state (9 numbers, use 0 for blank): "
).split()))

goal = tuple(map(int, input(
    "Enter goal state (9 numbers, use 0 for blank): "
).split()))

if (len(initial) != 9 or len(goal) != 9
        or set(initial) != set(range(9))
        or set(goal) != set(range(9))):
    print("Invalid input! Enter each number from 0 to 8 exactly once.")
else:
    astar(initial, goal)
```

OUTPUT:

```
Enter initial state (9 numbers, use 0 for blank): 1 2 3 5 0 6 4 7 8
Enter goal state (9 numbers, use 0 for blank): 1 2 3 4 5 6 7 8 0

Goal reached!
Total moves: 4
Total steps: 5
Step: 0
g(n) = 0
h(n) = 4
f(n) = 4
(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

Step: 1
g(n) = 1
h(n) = 3
f(n) = 4
(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

Step: 2
g(n) = 2
h(n) = 2
f(n) = 4
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)
```

```
...  Step: 2
     g(n) = 2
     h(n) = 2
     f(n) = 4
     (1, 2, 3)
     (4, 5, 6)
     (0, 7, 8)

     Step: 3
     g(n) = 3
     h(n) = 1
     f(n) = 4
     (1, 2, 3)
     (4, 5, 6)
     (7, 0, 8)

     Step: 4
     g(n) = 4
     h(n) = 0
     f(n) = 4
     (1, 2, 3)
     (4, 5, 6)
     (7, 8, 0)
```

ii) Implement A* Search Using Manhattan Distance:

```python
import heapq

def print_grid(state):
    for i in range(0, 9, 3):
        print(state[i:i + 3])
    print()

def manhattan_distance(state, goal):
    distance = 0

    for i in range(9):
        tile = state[i]

        if tile != 0:
            row, col = divmod(i, 3)
            goal_pos = goal.index(tile)
            goal_row, goal_col = divmod(goal_pos, 3)

            distance += abs(row - goal_row)
```

```
            distance += abs(col - goal_col)

    return distance

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            new_state = list(state)

            new_state[zero], new_state[new_pos] = (
                new_state[new_pos], new_state[zero]
            )

            neighbors.append(tuple(new_state))

    return neighbors

def astar(initial, goal):
    pq = []
    h = manhattan_distance(initial, goal)
    heapq.heappush(pq, (h, 0, initial))

    parent = {initial: None}
    cost = {initial: 0}

    while pq:
        f, g, current = heapq.heappop(pq)

        if g != cost[current]:
            continue

        if current == goal:
            path = []

            while current is not None:
                path.append(current)
```

```
                current = parent[current]

            path.reverse()

            print("\nGoal reached!")
            print("Total moves:", len(path) - 1)
            print("Total steps:", len(path))

            for step, state in enumerate(path):
                g_value = step
                h_value = manhattan_distance(state, goal)
                f_value = g_value + h_value

                print("Step:", step)
                print("g(n) =", g_value)
                print("h(n) =", h_value)
                print("f(n) =", f_value)
                print_grid(state)

            return

        for neighbor in get_neighbors(current):
            new_g = g + 1

            if neighbor not in cost or new_g < cost[neighbor]:
                cost[neighbor] = new_g
                parent[neighbor] = current

                h = manhattan_distance(neighbor, goal)
                new_f = new_g + h

                heapq.heappush(
                    pq, (new_f, new_g, neighbor)
                )

    print("No solution found.")

initial = tuple(map(int, input(
    "Enter initial state (9 numbers, use 0 for blank): "
).split()))

goal = tuple(map(int, input(
    "Enter goal state (9 numbers, use 0 for blank): "
).split()))
```

```
if (len(initial) != 9 or len(goal) != 9
        or set(initial) != set(range(9))
        or set(goal) != set(range(9))):
    print("Invalid input! Enter each number from 0 to 8 exactly once.")
else:
    astar(initial, goal)
```

OUTPUT:

```
Enter initial state (9 numbers, use 0 for blank): 1 2 3 5 0 6 4 7 8
Enter goal state (9 numbers, use 0 for blank): 1 2 3 4 5 6 7 8 0

Goal reached!
Total moves: 4
Total steps: 5
Step: 0
g(n) = 0
h(n) = 4
f(n) = 4
(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

Step: 1
g(n) = 1
h(n) = 3
f(n) = 4
(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

Step: 2
g(n) = 2
h(n) = 2
f(n) = 4
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)
```

```
Step: 3
g(n) = 3
h(n) = 1
f(n) = 4
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step: 4
g(n) = 4
h(n) = 0
f(n) = 4
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```